



Runtime Application Self-Protection (RASP)

CEN429 Secure Programming — Week 6

Asst. Prof. Dr. Uğur CORUH · 23.10.2026

Today

Hour	Section	Topic
1	1–3	What RASP is, RASP architecture · Integrity checking (self-hashing) Demo 1
2	4–6	Debugger detection Demo 2 · Environment/emulator detection Demo 3 · Hook/instrumentation detection Demo 4
3	7–10	Memory protection · Root/signature verification Demo 6-7 · Control-flow integrity Demo 5 · Response policy Demo 8 · Project

Learning outcome (LO.3): recognise RASP checks (integrity, anti-debug, environment, hooking) · design a response policy and device binding · explain RASP's limits and its ethical framework

RASP = **Runtime Application Self-Protection**: the application **monitoring** itself while it runs and **responding** to threats. Detect → defend → deter. But never a single check — a **layer**.

What We Bring from Earlier Weeks

- **The white-box attacker model** — the user who owns the device may also be the attacker; they can read memory and attach a debugger (**Week 1**)
- **Digest, MAC, and HMAC** — a digest is a one-way fingerprint computed from data; HMAC proves a message hasn't changed using a shared secret key (**Week 3**)
- **Deriving keys from a master secret (HKDF)** — deriving new keys from a master secret through Extract and Expand steps (**Week 3**)
- **Device binding** — a secret being meaningful only on a specific device, the innermost layer of the security shell (**Week 3**)

This week: the white-box model → the **MATE** attacker model (Section 1); HMAC → verifying the application's own code (Section 3); HKDF → deriving a key from a device fingerprint (Section 10); device binding → part of a RASP response policy (Section 10).

This Week's Concepts

Each term is defined once, where it first appears in the body; here we only mark **where**.

Concept	Where
RASP (detect/defend/deter)	Section 1
The MATE attacker model	Section 1
RASP architecture (when/where/how)	Section 2
Integrity / self-hashing	Section 3
Debugger detection	Section 4
Emulator/VM detection	Section 5
Hook / LD_PRELOAD detection	Section 6
Dynamic memory protection	Section 7
Root indicator	Section 8
Component signature verification	Section 8
Control-flow counter	Section 9
Response policy and device binding	Section 10



1. What Is RASP?

A Brief History — the Application Protects Itself

- **1980s** — the crack/anti-crack culture: the origin of **anti-debug** and self-checks
- **2000s** — DRM and mobile banking move protection **inside** the application
- **2012** — Gartner coins the term **RASP**
- **today** — **OWASP MASVS-RESILIENCE** turns this into an auditable requirement

RASP is not a new idea: it is the name of the corporate answer to the **MATE attacker**.

WAF ↔ RASP — Diagram

WAF ile RASP farkı

“kurcalandım mı, beni kim izliyor, hangi cihazdayım?”

WAF

uygulamanın ÖNÜNDE
HTTP isteklerini süzer

uygulamanın içini bilmez
sunucu tarafı

RASP

uygulamanın İÇİNDE
kendi kodunu, belleğini,
ortamını izler

MATE saldırganına cevap

RASP, cihazın güvenilmez olduğu durumlar için tasarlanmıştır.

WAF ↔ RASP Comparison

Protection	Where It Stops	What It Sees
Network firewall	At the network edge	Packets
WAF (Web App Firewall)	In front of the application	HTTP requests
RASP	Inside the application	Its own code, memory, environment

A WAF asks "is the incoming request malicious?"; RASP asks "have **I** been tampered with?"

The Attacker Model: MATE

- **MATE (Man-At-The-End):** an attacker who **owns the endpoint the application runs on**. They read memory, modify code, and trace it with a debugger. Identical to the **white-box** model from Week 11.
- **The hard truth:** if the device's owner is the attacker, **no client-side protection is ultimately unbreakable**.
- So why does RASP exist? The goal is not **unbreakability**; it's making the attack **unscalable and expensive**.

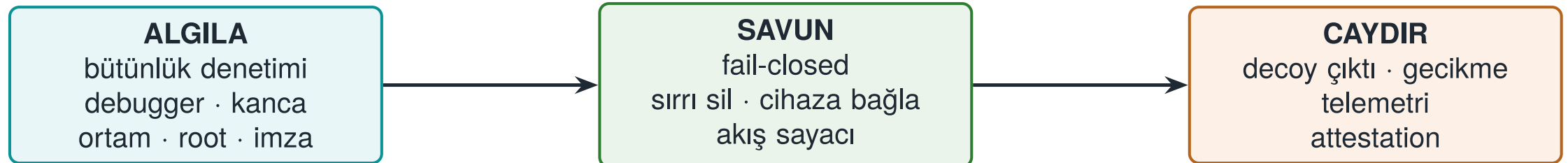
Three Jobs: Detect, Defend, Deter

- **Detect:** sense the abnormal environment/behaviour (debugger, emulator, tampering).
- **Defend:** if there's a threat, restrict/halt the feature.
- **Deter:** make the attack costly → the attacker gives up.

The RASP Loop — Detect → Defend → Deter

RASP döngüsü: algıla → savun → caydır

koruma uygulamanın İÇİNDE çalışır



Tek denetim değil, birbirini kontrol eden ÖRTÜŞEN AĞ → tek yama yetmez.

What Does RASP Complete?

- Static protections (Weeks 4–5) apply **without running**.
- Obfuscation (Weeks 9/11/14) makes **understanding** harder.
- RASP adds active defence **while the program runs**.

Obfuscation hides RASP; RASP protects obfuscation while it runs.

Why Is RASP Necessary?

- A white-box attacker **runs** the program and observes it (debugger, Frida).
- Static obfuscation alone does not stop this.
- RASP checks "am I being watched?" and responds.

The Main Rule (Up Front)

RASP is not unbreakable on its own; it gains strength from **layered** and **varied** checks.

A single "is it rooted?" check is easily bypassed; dozens of different checks together are hard.



2. RASP Architecture

When Does a Check Run?

- **At startup:** environment check on launch.
- **Right before a critical operation:** immediately before payment/key use.
- **Periodically:** regularly, in the background.
- **Randomly:** at unpredictable moments (so the attacker can't time it).

Where Does a Check Run?

- **On the native side** (C/C++): harder to reverse-engineer, preferred.
- **On the managed side** (Java): easier to read, but still necessary.
- **Cross-checking**: the two sides check each other.

The Cross-Check Idea

- The Java side checks the native library's **digest**.
- The native side checks the Java side's **integrity**.
- The attacker has to bypass **both at once**.

Multi-Point Placement and Decoy Parameters

- **Multi-point placement:** the same check done in **many places** in the code, each time in a slightly different form; when one point is patched, the others still run.
- **Inside the protected function itself:** a valuable function (e.g., key derivation) performs **several checks of its own** at its entry point — in the field this can reach **close to fifteen** checks.
- **Decoy parameters:** at an entry point, there can be extra parameters, hidden for readability, that trigger an integrity check (Week 4).

How Does a Check Run? (Hidden)

- The check code is **obfuscated** (Week 9): so the attacker can't easily find it.
- The result is **opaque** (not plain true/false).
- The response can be **delayed** (more on this shortly).

Weak Design — a Single Decision Point

```
if (debugger_present() || is_rooted() || integrity_broken())  
    return ERROR;          /* patching this one line  
                           bypasses everything */  
decrypt_key();
```

An attacker who flips a single comparison bypasses **the entire protection**.

Strong Design — Data Dependency

The check result feeds into **the data of the next computation**, not into an `if`:

- Check results feed into **key derivation** (the correct key if clean, a different but plausible-looking key if not).
- Check results are carried with **opaque values** (Week 4).
- The response happens **far in time and code** from the trigger.

A Layered Network of Checks

- Not a single check, but **dozens** of different checks.
- A **network** that checks itself (a checker network).
- Bypassing one triggers another.

Layers Protect Each Other — Diagram

Katmanlar birbirini korur

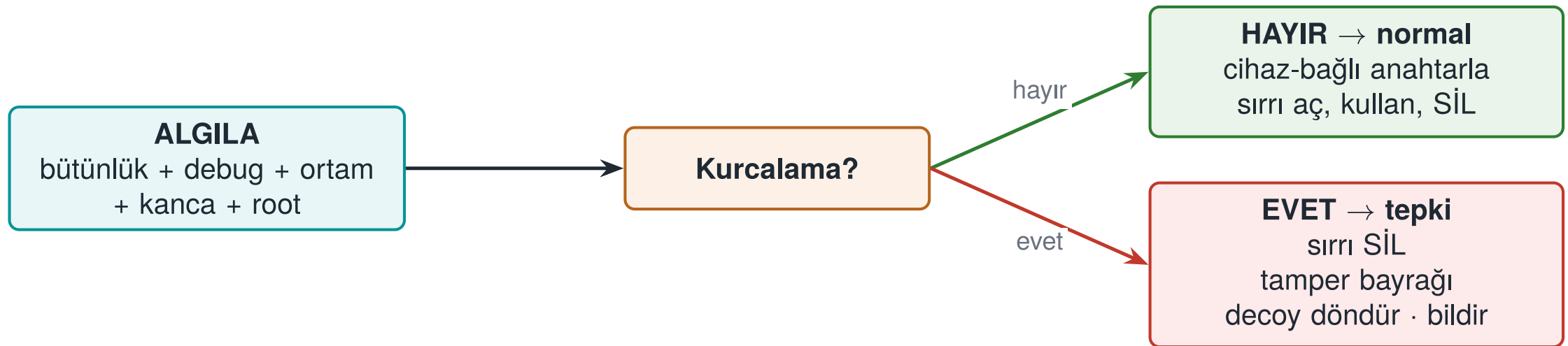
aynı hassas işlemi saran katmanlar



Architecture · Summary Diagram

RASP motoru: her kritik işlemde

algıla → karar ver → davran



Tepki gecikmeli/sessiz olmalı: hemen çökmek saldırgana denetimin yerini gösterir.

This Section's Rule (1–2)

Do not do the check **in one place, at one moment, with one if** .

Multi-point, cross-checked, hidden, and data-dependent checks **together** build a RASP architecture.

Section 1–2 — Quick Check

1. RASP's three jobs?
2. Why is cross-checking strong?
3. Why a network instead of a single check?

Section 1–2 — Answers

1. **Detect** (tampering/debugging/emulator), **Defend** (protect the key/data, restrict the feature), **Deter** (response: shut down/break/notify).
2. Checks **watch each other**; when the attacker disables one, another notices → silencing a single point isn't enough.
3. A single check is bypassed with **a single patch**; an overlapping network requires breaking every node separately → it raises **the cost of the attack**.



3. Integrity Checking (Self-Hashing)

The Idea

- The program computes a digest of **its own code sections**.
- It compares it against the expected digest.
- If it's been tampered with, the digest **won't match** → response.

How Does It Work? · Step by Step

1. After compilation, the digest of the critical code blocks is **recorded**.
2. While running, the program **recomputes** the digest of the same blocks.
3. It compares it against the recorded digest.
4. If there's a difference → tampering → response.

Step 1 · Record the Digest

- **After** compilation finishes, the critical code/data region is identified.
- This region's digest is computed and stored as the **golden value**.
- This value is the signature of the "untampered" state.

Step 2 · Recompute While Running

- **While running**, the program **recomputes** the digest of the same recorded code/data region.
- This computation happens not at compile time, but at **runtime**.
- It can be done not just once, but **periodically**.

Step 3 · Compare

- The newly computed digest is compared against the recorded **golden value**.
- The comparison must be done in **constant time** (not `memcmp`): a plain comparison stops at the first differing byte and leaks timing (Week 3).
- If equal: integrity is **fine**.

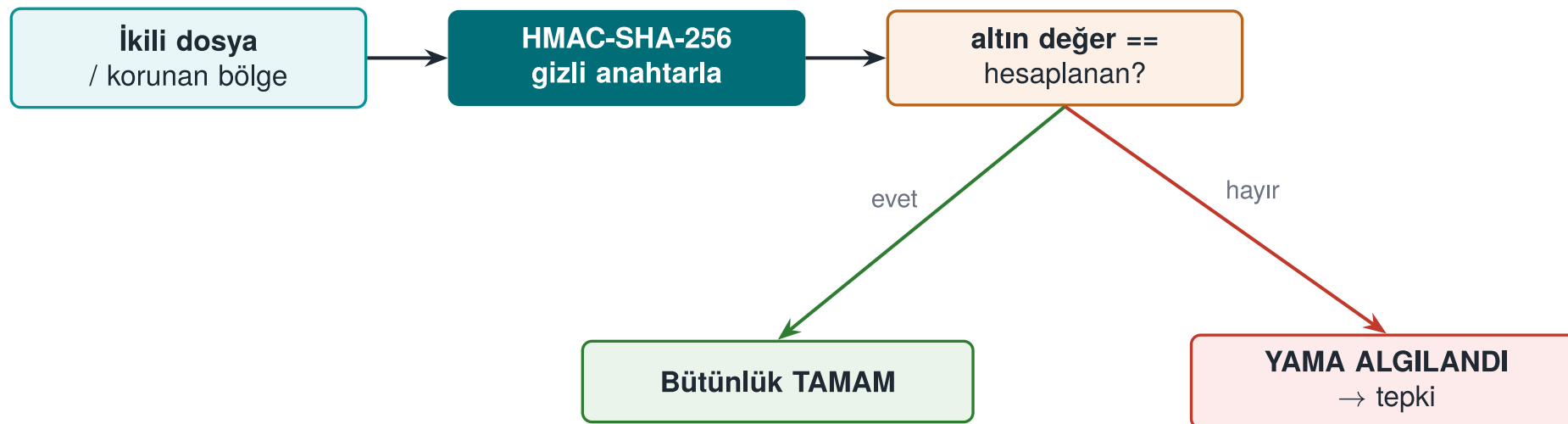
Step 4 · Respond

- If the digest **doesn't match** → tampering (a patch) has been detected.
- The result is tied not directly to an `if`, but to a **response** (Section 10).
- A smarter response is preferred over "crash immediately."

Self-Hashing — Diagram

Bütünlük denetimi (self-hashing)

uygulama kendi baytlarını doğrular



Tek başına yetmez: denetimin kendisi de yamalanabilir → örtüşen denetim gerekir.

What Does It Catch?

- A **patch** applied to the binary file.
- Bytes changed to skip a check.
- Injected code.

Demo 1 · Runtime Integrity Checking

`code/week-06/01-integrity-hmac` — self-hashing, HMAC-SHA-256.

- The textbook (Viega & Messier, Recipe 12.2) does this with **CRC32**; **CRC32 is not cryptographic**, an attacker can easily adjust bytes to make the CRC match. We use **HMAC-SHA-256** (needs a secret key).
- The program computes its own binary file's digest and records it as the **golden value**.
- A **copy** of the binary file is made and one byte in it is changed (a patch simulation); the patched copy's digest is compared against the golden value.

Demo 1 · The Core of the Code

```
/* Find own path, read the file, compute HMAC */
self_path(path, sizeof(path));
read_file(path, &data, &size);
crypto_hmac_sha256(RASP_KEY, 32, data, size, digest);

/* constant-time comparison */
unsigned char diff = 0;
for (int i = 0; i < 32; i++)
    diff |= expected[i] ^ digest[i];
if (diff == 0) { /* OK */ } else { /* PATCH */ }
```

Integrity check (animation)

code bytes =

0	1	2	3	4	5	6	7	8	9	10	11
7F	45	4C	46	02	01	01	00	90	83	04	08

digest (acc) =

C8A921BE

golden value saved

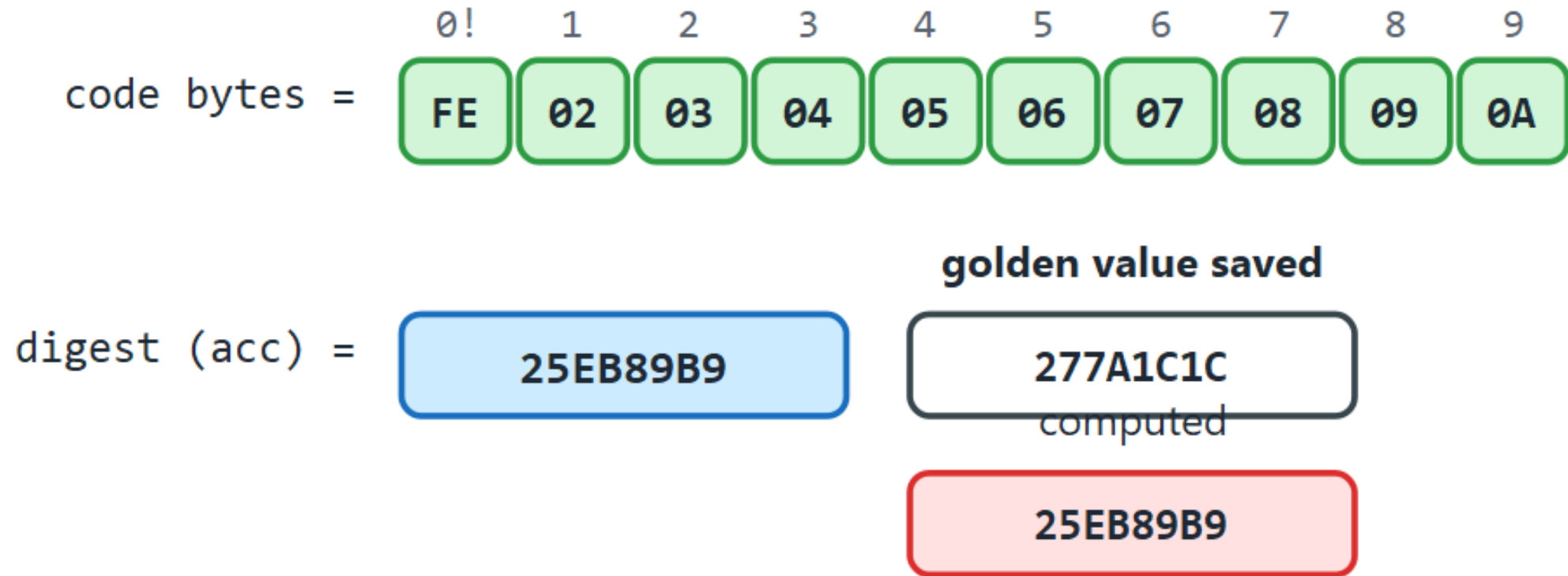
C8A921BE

computed

C8A921BE

0xC8A921BE == 0xC8A921BE -> RESULT: INTEGRITY OK.

Integrity check — edge case: first byte patched



0x277A1C1C != 0x25EB89B9 -> RESULT: PATCH DETECTED.

Demo 1 · Actual Output

STEP 2 - Normal run: integrity verifies OK (no patch)

Expected : 5aeaf45226dbb29af7...b55115de4

Computed : 5aeaf45226dbb29af7...b55115de4

RESULT: INTEGRITY OK - target unchanged.

STEP 3 - Attack: 1 byte of the copy is changed

STEP 4 - The patched copy's HMAC is compared

Expected : 5aeaf45226dbb29af7...b55115de4

Computed : c47c35687dc205b93a...aca0ec8a

RESULT: PATCH DETECTED - target was modified!

Changing a single byte completely changed the digest (the avalanche effect).

Self-Hashing Alone Is Not Enough

- The attacker can find the **digest function** and force it to always say "match."
- Or bypass the check with hardware breakpoints (van Oorschot et al.).
- So: hide it, diversify it, cross-check it.

Strengthening It

- The digest-checking code is **hidden** (Week 9).
- **Multiple** blocks, overlapping regions.
- The result is not a plain comparison, it **feeds into a computation** (the check result affects the program's behaviour).

Overlapping Checks

- One check also covers another check's code.
- When the attacker disables one, the other **breaks**.
- Bypassing from a single point becomes hard.

This Section's Rule (3)

Verify the integrity of critical code/data regions at runtime with a **cryptographic** digest (HMAC/signature), not CRC.

Don't do the check at a single point and only at startup; make it **periodic, multiple, and overlapping**.



4. Debugger Detection

Anti-Debug Approaches — Diagram

Hata ayıklayıcı algılama yolları

hiçbiri tek başına kesin değil — sinyaller birlikte tartılır

İşletim sistemi bayrağı

Linux `/proc/self/status` TracerPid · Windows `IsDebuggerPresent`

PEB / NtGlobalFlag

Windows süreç yapısındaki izler

Zamanlama

iki nokta arası süre eşiği aşıyor mu?

Kendi kendini izleme

`ptrace(PTRACE_TRACEME)` başarısızsa biri izliyor

Her sinyal atlatılabilir; güç, birden çok bağımsız sinyalin birlikteliğinden gelir.

Why Is a Debugger Dangerous?

- **Debugger:** a tool that runs a program step by step, halting it and reading its memory (gdb, lldb, x64dbg, WinDbg).
- The attacker **halts** the program, reads memory, changes values.
- They can bypass checks one by one.
- RASP checks "is a debugger attached to me?"

Detection Approaches for a Debugger (Concept)

- Asking the operating system for "am I being traced?" information.
- The state of specific debugging interfaces.
- Timing: if an operation took **much longer** than expected, someone may be single-stepping it.

Indicators by Platform · What's Outdated

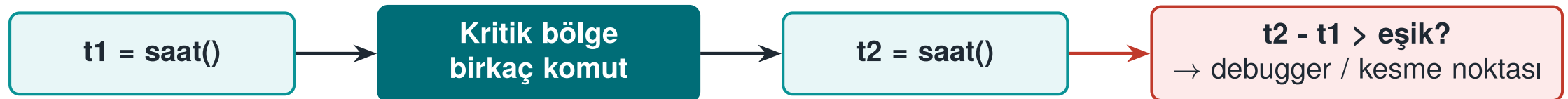
Environment	Indicator
Linux / WSL	<code>/proc/self/status</code> → <code>TracerPid</code> (if > 0, we're being traced); is the parent process <code>gdb/strace</code>
Windows	<code>IsDebuggerPresent</code> (PEB <code>BeingDebugged</code>); <code>CheckRemoteDebuggerPresent</code> ; PEB <code>NtGlobalFlag</code>

From 2003 to today: SoftICE is dead, replaced by x64dbg/WinDbg/gdb/lldb/**Frida**; `ptrace` + **TracerPid** is still valid.

Timing-Based Detection

Zamanlama tabanlı sezme

debugger kodu yavaşlatır



Yanlış pozitif riski: yavaş cihaz, arka plan yükü. Eşiği ölçerek seçin.

Debugger single-stepping **slows down** the work.

Demo 2 · Debugger Detection

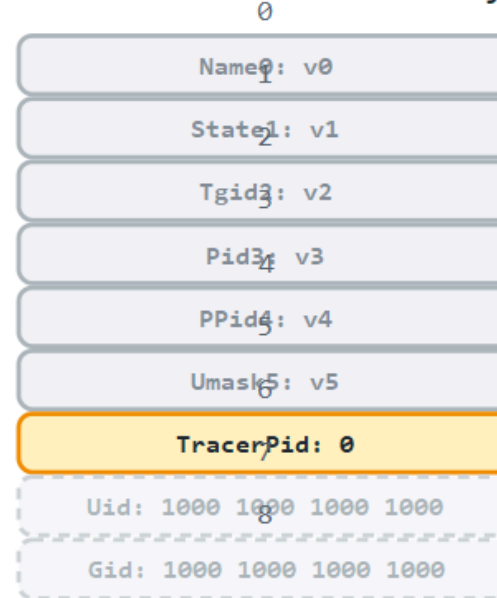
`code/week-06/02-debugger-detection` — anti-debug, read-only.

- The program looks at several independent signals and **changes nothing**.
- It doesn't even self-`ptrace`; it only **reads** `/proc` and PEB fields.
- `demo.sh` runs the program first normally, then under `gdb`.

```
/* Read TracerPid from /proc/self/status */  
long tp = tracer_pid();  
if (tp > 0) suspicious++; /* a process is watching us */
```

Debugger detection (animation)

/proc/self/status is scanned line by line:



TracerPid=0 -> no tracer (clean)

parent process name =

bash

no known word -> clean

RESULT: clean - no tracing tool visible.

Debugger detection — edge case: gdb attached

/proc/self/status is scanned line by line:

Name	0	v0
State	1	v1
Tgid	2	v2
Pid	3	v3
PPid	4	v4
Umask	5	v5
Ngid	6	v6
NStgid	7	v7
TracerPid	8	7780
Uid	9	1000 1000 1000 1000
Gid	10	1000 1000 1000 1000

TracerPid=7780 -> TRACED (suspicious)

parent process name =

gdb

matched: "gdb" -> SUSPICIOUS

RESULT: a debugger/tracing TOOL was detected (2 signal(s)).

Demo 2 · Actual Output (WSL)

STEP 1 - Normal run (NO debugger)

1) /proc/self/status TracerPid : 0 (no tracer)

2) Parent process name : sh

RESULT: clean - no tracing tool visible.

STEP 2 - Run under gdb

1) /proc/self/status TracerPid : 41885 (TRACED)

RESULT: a debugger/tracing TOOL was detected (2 signal(s)).

Without a debugger, TracerPid is **0**; under gdb it came out **41885** (>0).



The Detection–Counter-Detection Race

Anti-debug is a classic **arms race**: the attacker can turn `ptrace` into a no-op with `LD_PRELOAD`, patch `IsDebuggerPresent`'s return value, or clear the PEB flag.

- Every single method is known and **can be bypassed**.
- Strength: **many** different methods + obfuscation + diversification.
- The goal is still **deterrence** and **delay**.

What About the Response?

- Crashing immediately is not always good (it tells the attacker "there's a check here").
- Better: a **delayed, indirect** response (Section 10, shortly).

This Section's Rule (4)

Don't write anti-debug as **a single** `if (IsDebuggerPresent()) exit();` .

Collect multiple independent signals, tie the result to a behaviour, **delay** the response.

Section 3–4 — Quick Check

1. What does self-hashing catch?
2. Why isn't it enough alone?
3. How does timing give away a debugger?

Section 3–4 — Answers

1. **Code/binary bytes being changed** (a patch, breakpoint/code injection) — it computes its own digest at runtime and compares it against the expected one.
2. The attacker can find/bypass the hash routine or the expected value, or NOP out the comparison; it also only catches a **static** change. Overlapping checks are needed.
3. A debugger/breakpoint **slows down** the code; if the elapsed time between two points crosses a threshold, debugging is sensed.



5. Environment Detection: Emulator/VM

Emulator False Positive — Diagram

Emülatör algılamada yanlış pozitif

suçsuz kullanıcıyı engellemek en pahalı hatadır

Zayıf sinyaller

pil yok · sensör yok
hipervizör biti

WSL2 / Hyper-V / VBS yüzünden
MEŞRU makinede de görülür

Doğru yaklaşım

birden çok sinyali TART
sertliği derecelendir

sunucu tarafı attestation
ikili “çalış/çalışma” değil

Bir skor üretin; eşiği ürünün risk iştahına göre ayarlayın.

Why an Emulator?

- **Emulator/VM:** a software environment that **imitates** a device (an Android emulator, QEMU).
- The attacker does their analysis **in an emulator** instead of a real device.
- Halting, tracing, and resetting is easy in an emulator.
- RASP checks "am I on a real device?"

Detection Clues (Concept)

- Hardware/sensor gaps specific to emulators.
- Typical fake identifiers.
- The absence of features expected on a real device.

Detection Techniques (Concrete)

Technique	How	Note
CPUID hypervisor bit	<code>CPUID.1:ECX[31]</code> — "hypervisor present"	0 on bare metal
Hypervisor vendor signature	CPUID leaf <code>0x40000000</code> → 12 characters	Some hypervisors hide it
Timing	Measure an operation; heavy emulation slows it down a lot	Threshold varies by machine

Demo 3 · VM/Emulator and Timing

`code/week-06/03-environment-timing` — only reads a CPU instruction and the clock.

```
(A) CPUID.1:ECX[31] hypervisor bit    : PRESENT
    Hypervisor vendor signature      : "Microsoft Hv"
(B) time for 2,000,000 iterations      : 0.662 ms
RESULT: a hypervisor was SEEN.
```

This output was captured on a **real** Windows laptop (Hyper-V).

VM/emulator detection (animation)

(A) CPUID leaf 1 — the ECX register

0x00000000

bit 31 (hypervisor bit)

(B) timing measurement (informational only)

elapsed

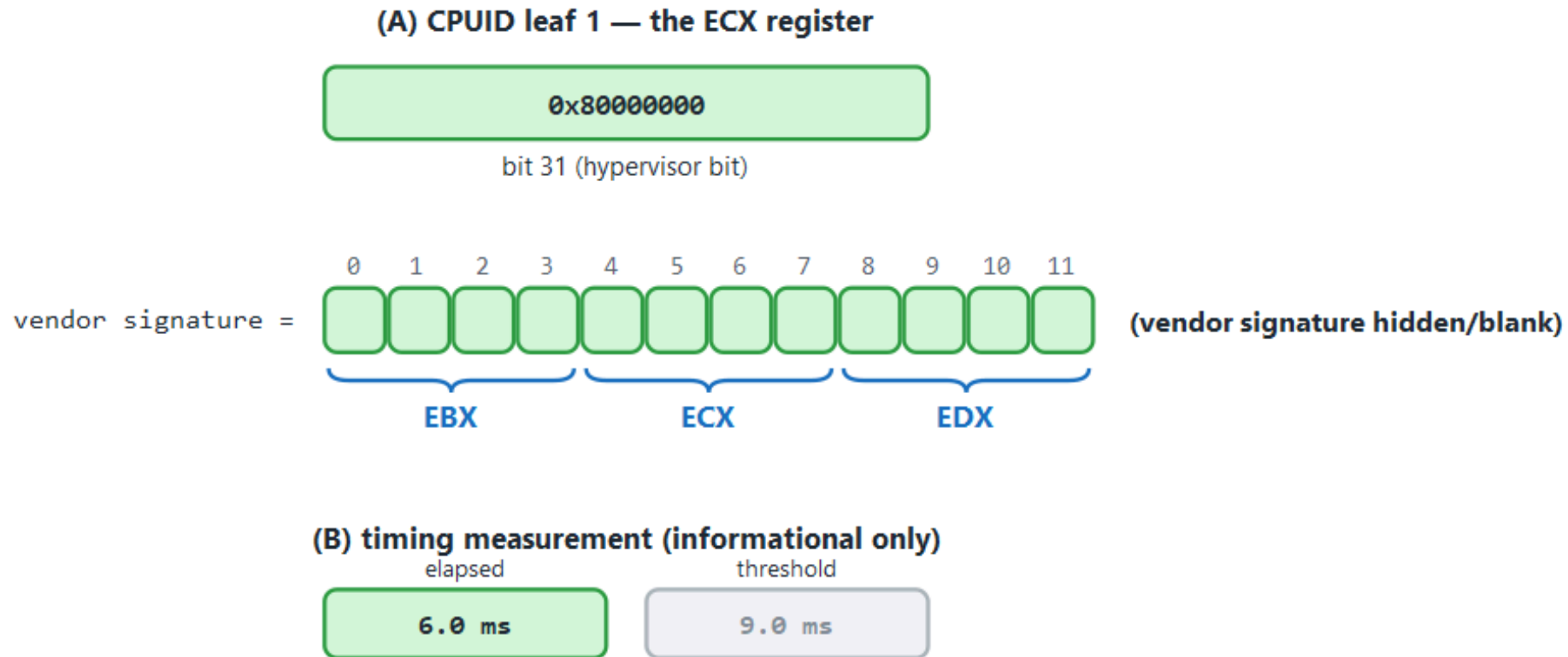
4.2 ms

threshold

9.0 ms

RESULT: no hypervisor visible (may be bare-metal).

VM/emulator detection — edge case: blank vendor signature



RESULT: a hypervisor was SEEN. WSL2/Hyper-V/VBS also look like this — this alone does not mean "analysis environment".

The Most Important Lesson: False Positives

- The output above said the hypervisor **"IS PRESENT"** AND the vendor signature is **visible** (`"Microsoft Hv"`).
- But this is **not** an analysis environment — it's an ordinary developer machine!
- On modern Windows, most machines report a hypervisor because of **Hyper-V, WSL2, VBS**.

The Cost of a False Positive

- This bit alone does **not** mean "an attacker is analysing."
- If an application **refused** to run just because it saw a hypervisor, it would block millions of **legitimate** users.
- Legitimate users: developers, enterprise VM users, ordinary Windows users with WSL2/Hyper-V/VBS.
- RASP always **weighs multiple indicators** and accepts that any single one **can be bypassed**.

Rule: a Risk Score, Not a Binary Decision

- Use environment detection as a **risk score**.
- **Combine** multiple indicators.
- Make the final decision together with a **server-side risk engine**.

How Is This Applied in the Field?

- In the mobile world the equivalent is **emulator detection**: `ro.kernel.qemu`, fake sensors, typical IMEI/serial values.
- The same principle applies: a signal, **not proof**.
- Combination + server-side verification.

This Section's Rule (5)

In emulator/VM detection, **a single bit never means "attack."**

Use a risk score instead of a harsh response; the result is evaluated together with server-side verification.



6. Hook and Instrumentation Detection

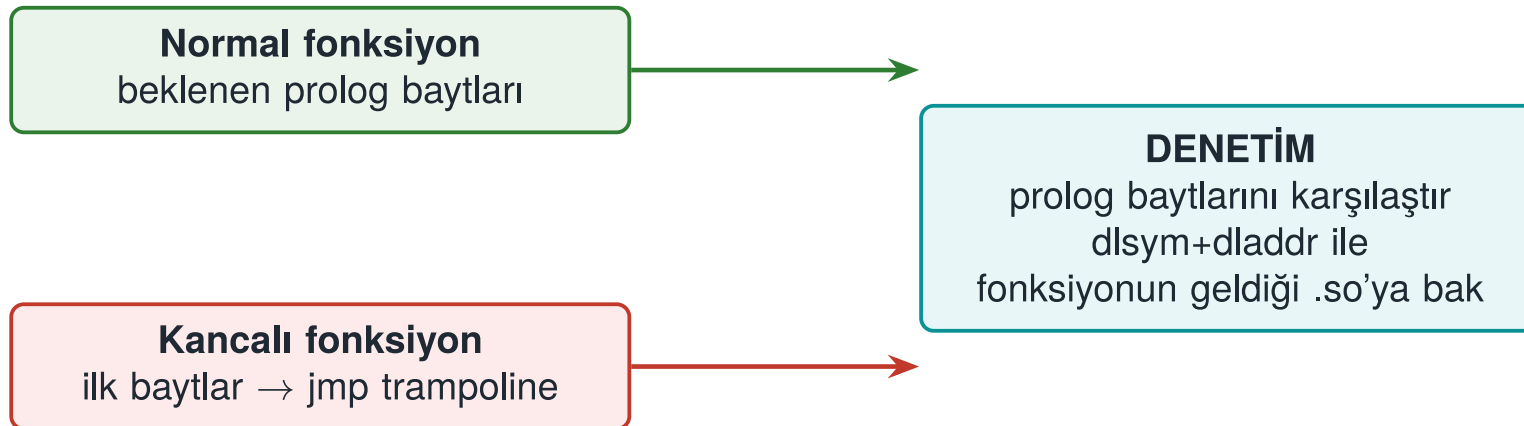
Brief History — From Static to Dynamic

- **1990s** — IAT/PLT-GOT hooks: hand-written, target-specific tools.
- **2000s** — `LD_PRELOAD` / DLL injection: general-purpose but **static**.
- **2013–2014** — **Frida** emerged: hook code is written **in JavaScript at runtime**, no compilation needed.

Hook Detection — Diagram

Kanca (hook) nasıl algılanır?

kanca, fonksiyonun ilk baytlarını değiştirir



libc/vDSO dışında bir modülden geliyorsa kanca vardır (LD_PRELOAD).

The Hook Threat

- The attacker **hooks** a critical function: intercepts and changes the call.
- Example: making the "is the signature valid?" function always return "yes."
- Tool: **Frida**, Xposed.
- **Instrumentation**: injecting code into a running program to observe/change its behaviour; Frida is the most common tool for this.

LD_PRELOAD Hooking

- On Linux, loading a library before the program to **replace** functions.
- The critical function is replaced with the attacker's version.
- RASP checks the loaded libraries.

Hooking Techniques — Table

Technique	How	Platform
LD_PRELOAD	Preloading a <code>.so</code> and replacing a function	Linux
PLT/GOT hooking	Redirecting a call-table entry	Linux/ELF
Inline hooking	Overwriting the first bytes with a <code>jmp</code>	Everywhere
Frida / Xposed	Dynamic binary instrumentation	Mobile/desktop

Detection Approaches for a Hook (Concept)

- Unexpected loaded libraries/modules.
- Traces of known instrumentation tools (in memory, on ports).
- Function addresses **deviating** from the expected.

A Weak Approach: getenv

```
if (getenv("LD_PRELOAD") == NULL) printf("temiz\n");
```

`getenv` itself can also be **hooked** to return `NULL`. That's why you need to **verify the source**:

1. **Resolve** the critical function (`dlsym`) → 2. find **which module it comes from** (`dladdr`) → 3. check whether it's **legitimate** → 4. **decide**.

Step 1 · Resolve the Function

```
void *p = dlsym(RTLD_DEFAULT, "time");
```

- `dlsym` finds, at runtime, **which function** would actually be called.
- If a preloaded hook exists, `dlsym` returns **that one** (a manipulated result).

Step 2 · Find the Source

```
Dl_info info;  
dladdr(p, &info); /* the .so that provides the function */
```

- `dladdr` tells you **which shared object (.so)** an address comes from.
- It does **not rely on** `getenv` -style spoofing — it looks directly at the memory address.

Step 3 · Check Whether It's Legitimate

```
int hook = !is_legitimate(info.dli_fname);
```

- The expected source is **libc**, the **vDSO**, or the dynamic loader.
- If it comes from a foreign **.so** → **there's a hook**.

Step 4 · Decide

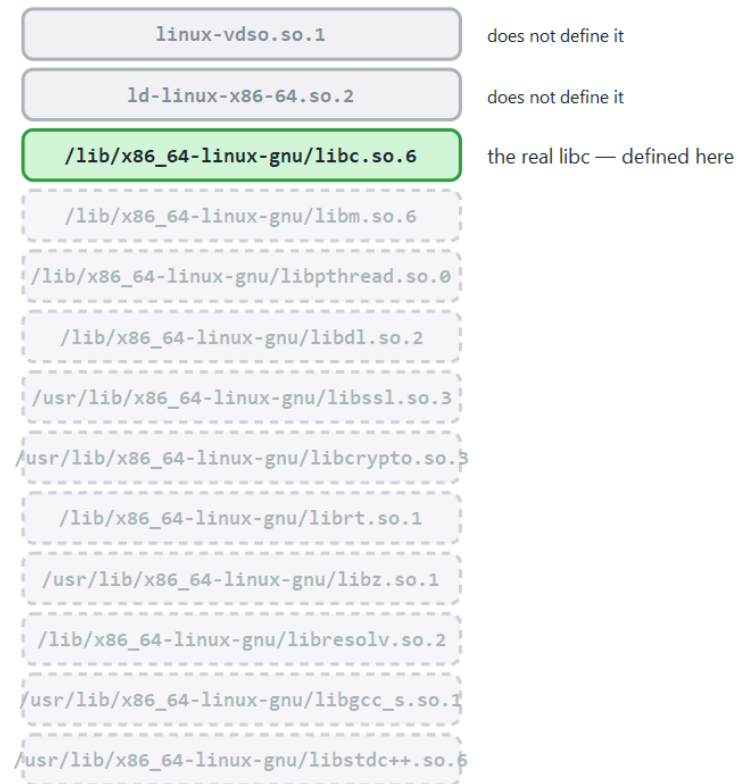
- If it's **not** in the list of legitimate sources → the hook flag is raised.
- The result isn't left alone; it's tied to the **response policy** (Section 10).
- It's **repeated** for multiple functions (a single function isn't enough).

Function Integrity

- Are the critical function's first bytes what's expected?
- A hook usually places a **jump** at the start → the bytes change.
- If they've changed → there's a hook.

LD_PRELOAD symbol resolution order (animation)

``dlsym(RTLD_DEFAULT, "time")`` tries loaded objects in order:



dladdr -> /lib/x86_64-linux-gnu/libc.so.6

RESULT: dladdr confirms the source is legitimate (libc/vDSO/loader) -> clean.

LD_PRELOAD order — edge case: hook right after vDSO

``dlsym(RTLD_DEFAULT, "getenv")`` tries loaded objects in order:



`dladdr -> /tmp/.evil/libfake_hook.so (HOOK!)`

RESULT: `dladdr` shows the source is NOT libc, but a preloaded hook -> HOOK DETECTED.

Demo 4 · LD_PRELOAD Hooking — Actual Output

```
STEP 1 - Normal run (no LD_PRELOAD)
time      -> linux-vdso.so.1
getenv    -> /lib/x86_64-linux-gnu/libc.so.6
RESULT: clean - no preload/hook visible.

STEP 2 - Attack: the fake hook is loaded ONLY into this process
time      -> bin/linux/libfake_hook.so (HOOK!)
time(NULL) returned : 1234567890 (fixed fake value)
RESULT: function hook / preload DETECTED (2 signal(s)).
```

In the clean run, `time` resolved from the kernel's **vDSO** — this is **legitimate**, not a hook.

Hook Detection Can Also Be Bypassed

- Advanced tools hide their traces.
- Strength: multiple methods + obfuscation + server-side verification.

This Section's Rule (6)

In hook detection, don't trust **manipulable** sources like `getenv(LD_PRELOAD)` .

Verify the **source** of critical functions (`dlsym` + `dlsym`); check multiple functions.



7. Dynamic Memory Protection

Brief History — "Only in RAM" Is Not Enough

- **2008** — Halderman and co-authors, in "**Cold Boot Attacks**," showed DRAM retains its contents for **seconds to minutes** after power loss; they recovered full-disk-encryption keys from memory by cooling a laptop and rebooting it.
- This is the **direct** motivation for the "short lifetime" and "never leave it in the clear" countermeasures.

Memory Protection Layers — Diagram

Dinamik bellek koruması: hangi katman neyi kapatır?

bellek saldırganının en kolay ulaştığı yerdir

Döküm kapatma

PR_SET_DUMPABLE=0 — çökme dökümünde sır kalmaz

Bellek kilidi

mlock — sır takas alanına yazılmaz

Kısa ömür + silme

explicit_bzero — pencere daralır

Bütünlük sayacı

korunan değer yanında sağlama tutulur; değişirse yakalanır

İzleme tespiti

ptrace / okuma denemesi algılanır ve puana katılır

Hiçbiri tek başına yetmez; her biri saldırganın bir yolunu pahalılaştırır.

The Problem

- Sensitive values (keys, counters) sit in memory.
- The attacker can read/change memory and bypass a check.

What Does the Attacker Do With Memory?

Attack	Example	Target
Reading	Dumping process memory, searching for a known value	Key, PIN, decrypted data
Changing	Changing a counter/flag in memory	Logic and checks
Watching	Tools that alert when a value changes	Discovering where variables live

All of these require the attacker to **access** process memory (debugger, OS interface, injected library).

The Protected Counter Idea

- Instead of keeping a critical counter in the plain:
 - keep it together with a **shadow copy** + a **digest**.
 - check consistency on every read/write.

Protected Value · Concept

```
typedef struct { uint32_t value; uint32_t shadow; uint32_t mac; } Protected;  
/* read: is value == ~shadow, and is mac correct? */
```

If the attacker changes only `value`, the inconsistency is caught.

Memory-Scanning Detection

- Is there unexpected access to sensitive regions being watched?
- Traces of a memory dump/scan.
- Goal: raise the cost of live memory analysis.

Memory Protection Is Limited

- A determined attacker can still get access.
- Goal: **delay**, make the key short-lived (Week 10).

This Section's Rule (7)

Keep things in memory **briefly, minimally, and scattered**; protect a critical value with a **shadow copy + digest**.

An inconsistency must be tied to a response — silently ignoring it defeats the protection.

Section 5–7 — Quick Check

1. Why is there a false-positive risk in emulator detection?
2. Why does a hook change function bytes?
3. What does a protected counter catch?

Section 5–7 — Answers

1. A legitimate user may also use a **VM/emulator/CI**; weak signals also show up on real devices → the risk of **blocking the innocent**. Grade the severity of the response.
2. A hook changes the target's **first bytes** with a jump (jmp/trampoline) so the call diverts into the attacker's code → **prologue bytes** differ from expected.
3. **A check in the control flow being skipped**: the counter increases as critical checks run; if it doesn't reach the expected value, a check has been **bypassed**.

8. Root Environment and Signature Verification

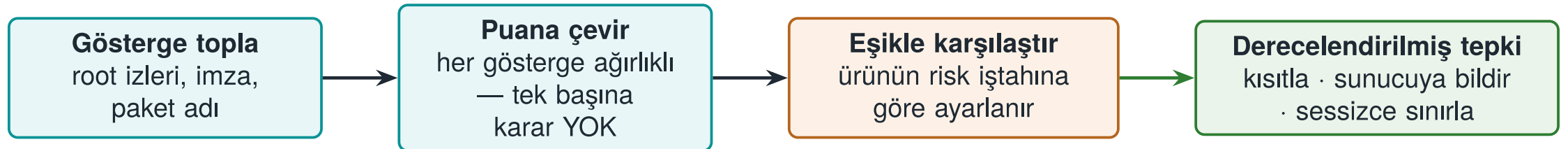
Brief History — Package Signing

- **Android 1.0 (2008)** — original APK signing (v1): a modified APK cannot carry the original signature.
- **Android 7.0 Nougat (2016)** — **APK Signature Scheme v2**: signs the whole APK, not file by file.
- **Android 9 Pie (2018)** — **v3** added **key rotation** support to the same principle.

From Root Indicator to Decision — Diagram

Kök/ayrıcalıklı ortam: göstergeden karara

tek göstergeyle engellemek meşru kullanıcıyı vurur



Root'lu cihaz suçlu değildir; yalnızca güvence düzeyi düşüktür. Tepkiyi ona göre ölçekleyin.

Why Is a Rooted Device a Risk?

- Root = full privilege.
- Protections weaken; the attacker gains access to memory, files, the process.
- RASP checks "is the device rooted?"

Root Detection (Concept)

- The presence of known root tools/files.
- Being able to perform operations that are normally restricted.
- System-integrity indicators.

Root Detection Can Be Bypassed

- Root hiding tools exist.
- A single indicator isn't enough; **many** indicators + server-side verification.
- Response: restrict the critical feature, not block everything (don't hurt the user).

Component Signature Verification

- Is the caller's/loaded component's **digital signature** the expected one?
- Catch it if a fake/modified application is calling the library.
- Example: only the main application with the expected signature can use the SDK.

Signature + Digest Together

- Signature: **who** published the component?
- Digest: **has** the component changed?
- Together they make forgery harder.

Root/Privilege Indicator Scan (animation)

privilege level =

normal user

2) dangerous-indicator scan:

- 1 /system/xbin/su
- 2 /system/bin/su
- 3 /sbin/su
- 4 /su/bin/su
- 5 /system/app/superuser.apk
- 6 /data/adb/magisk
- 7 /dev/socket/magisk
- 8 /data/local/tmp/frida-server
- 9 /system/bin/.ext/su
- 10 /cache/.disable_magisk
- 11 /system/bin/failsafe/su
- 12 /system/xbin/su
- 13 /system/usr/we-need-root/su-backup
- 14 /vendor/bin/su

0/14 found

RESULT: clean - no risky-environment indicator.

Root Indicator — edge case: multiple marks + elevated

privilege level =

root/admin

2) dangerous-indicator scan:

/system/xbin/su

/system/bin/su

/sbin/su

/su/bin/su

/system/app/superuser.apk

/data/adb/magisk

/dev/socket/magisk

/data/local/tmp/frida-server

/system/bin/.ext/su

/cache/.disable_magisk

/system/bin/failsafe/su

/system/xbin/su

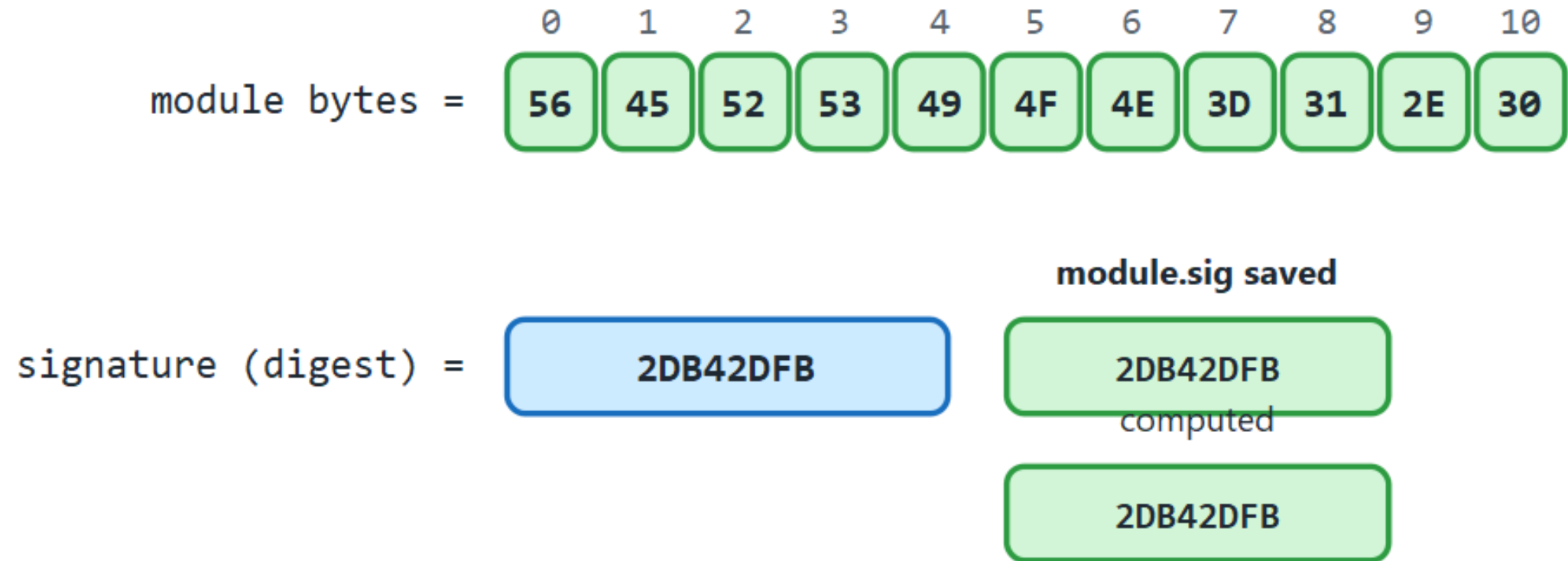
/system/usr/we-need-root/su-backup

/vendor/bin/su

4/14 found

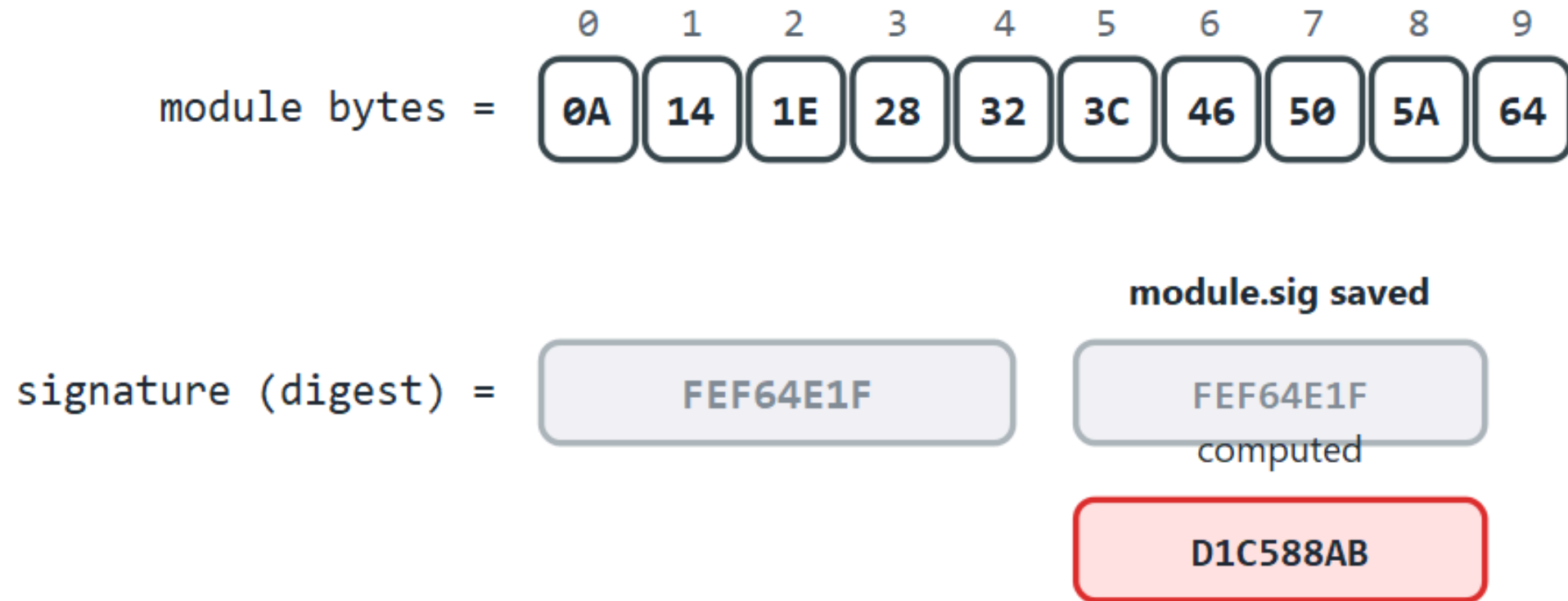
RESULT: an elevated/risky environment INDICATOR is present (5 signal(s)).

Component Signature Verification (animation)



0x2DB42DFB == 0x2DB42DFB -> the signature HELD, safe to load.

Component Signature — edge case: truncated module



0xFEf64E1F != 0xD1C588AB -> the signature DID NOT HOLD, loading REJECTED.

Demo 6 and 7 · Actual Outputs

```
# Demo 7 (root/privilege indicator)
Privilege level : normal user
[FOUND] output\fake_su (extra indicator)
RESULT: an elevated/risky environment INDICATOR is present.

# Demo 6 (component signature)
Module signature DID NOT HOLD -> REPACKAGED.
Loading REJECTED.
```

Both demos work by **only reading/verifying**; they don't change the system.

This Section's Rule (8)

Root/privilege indicators are **a signal, not proof** — they're easily spoofed.

Cryptographically verify every dynamically loaded component **before** loading it; set up mutual verification where possible.



9. Control-Flow Integrity

Brief History — From Tamper-Resistance to CFI

- **1996** — Aucsmith: distributed integrity checks that **cross-check** each other.
- **2001** — the Collberg/Thomborson taxonomy; Chang/Atallah's "**guard**" idea — multiple checkers.
- **2005** — Abadi, Budiu, Erlingsson, Ligatti formally defined **Control-Flow Integrity (CFI)**.

Why Isn't a Single `if` Enough?

```
if (signature_valid()) proceed();    /* single point */
```

- The attacker **patches** this single branch → the check is skipped.
- Week 9's opaque boolean + randomised exit is critical here.

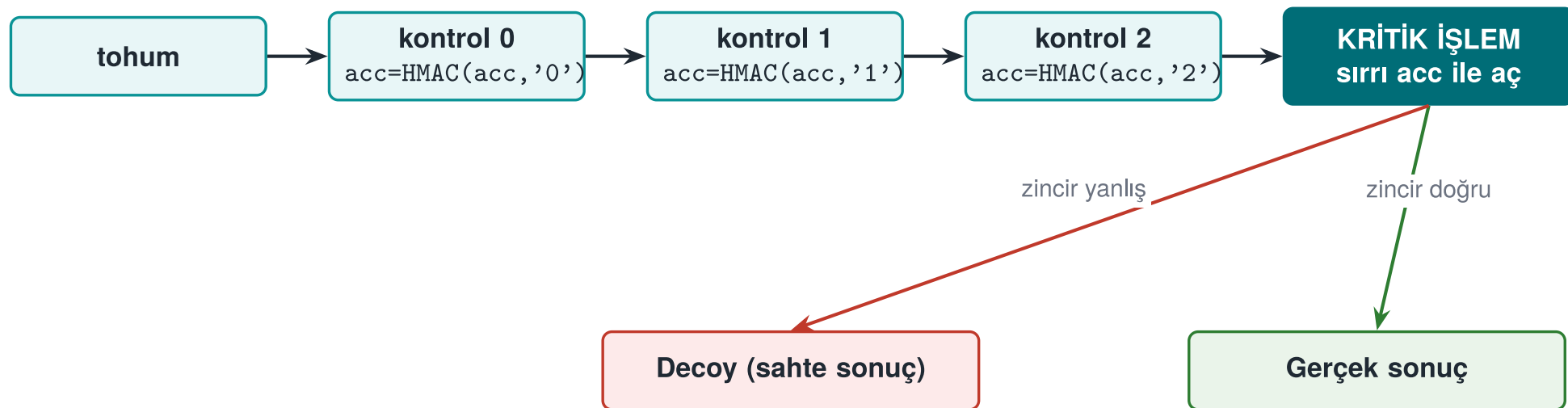
Control-Flow Counter

- Critical checks must pass in the correct **order**.
- Every check updates a **counter**.
- If the counter isn't the expected value at the end → one was skipped.

Control-Flow Counter — Diagram

Kontrol akışı sayacı: denetim atlandı mı?

her denetim akümülatörü ilerletir



Tek if bir NOP'la atlanır; sayaç, denetimin GERÇEKTEN çalıştığını sonradan kanıtlar.

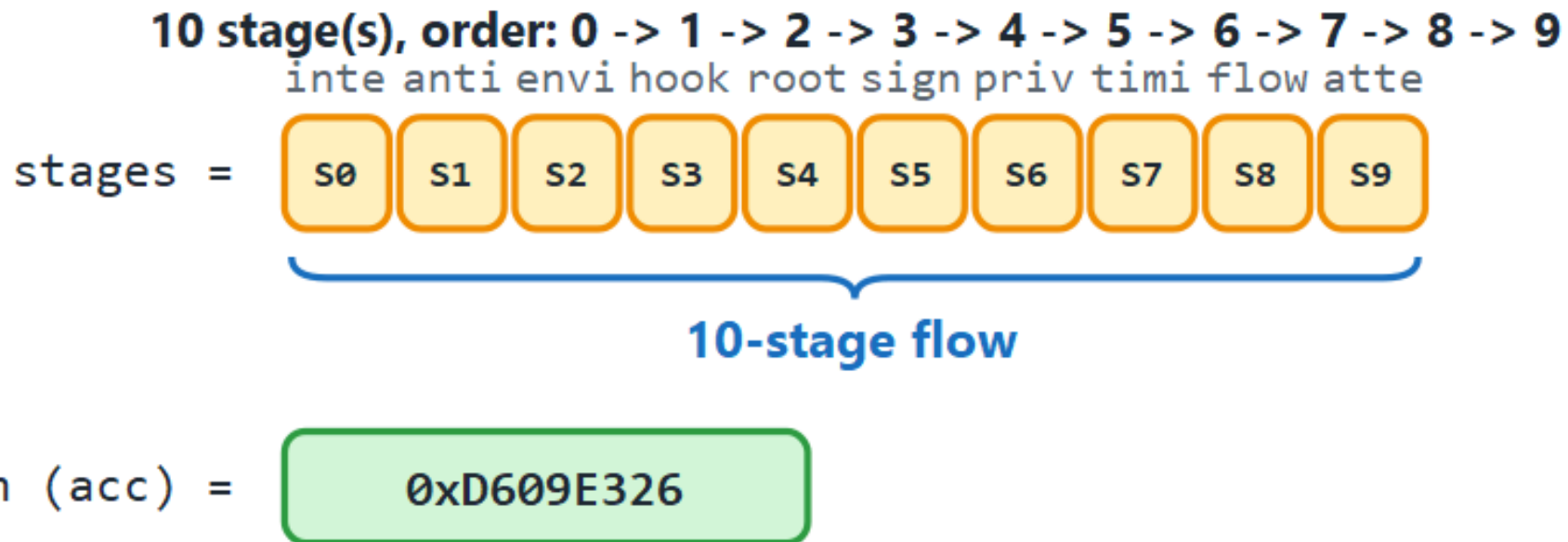
Dual Counters

- `count` (how many checkpoints ran) + `visited_mask` (which checkpoints ran, a bit set).
- Both are **separate** from the HMAC chain and give a readable diagnostic — but neither decides alone.
- Even if the attacker patches these, the real key still comes only from the **HMAC chain** (Demo 5).

Tie the Result to a Behaviour

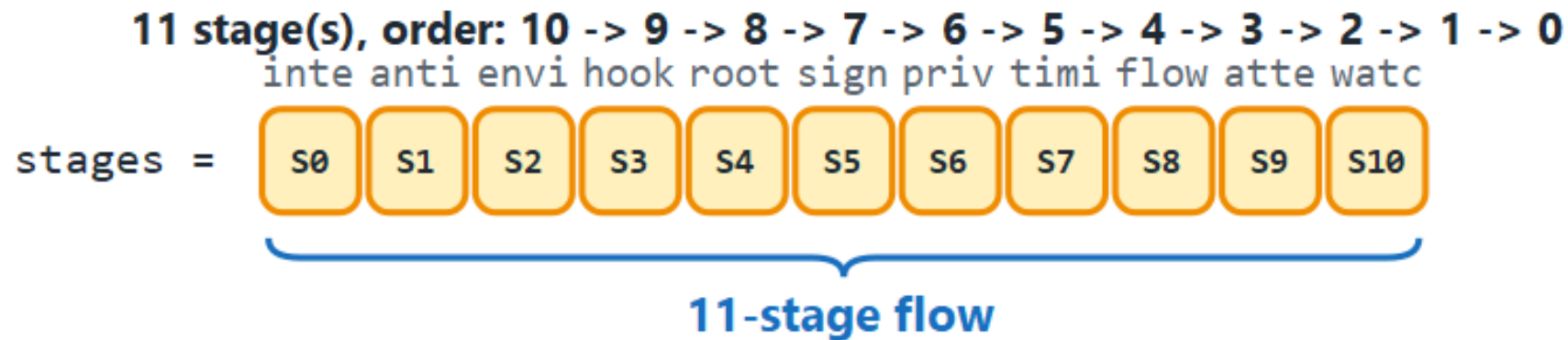
- The check result is not a plain `if`.
- The result feeds into **the computation** of the next step (e.g., a key derivation).
- If skipped, the program behaves **incorrectly** (breaks silently).

Control-Flow Counter (animation)



0xD609E326 == 0xD609E326 -> RESULT: the critical operation produces the right result (APPROVED).

Control-Flow Counter — edge case: wrong order



chain (acc) =

0x9D1A4917

0xB96C8707 != 0x9D1A4917 -> RESULT: the chain is wrong, the critical operation is DENIED (a decoy is returned).

Demo 5 · Skip Attack — Actual Output

```
SCENARIO 1 - normal: every checkpoint runs in order  
RESULT: PAYMENT APPROVED -> "PAYMENT-APPROVAL-TOKEN-4242"
```

```
SCENARIO 2 - skip: the checks are SKIPPED entirely  
  (warning: count=0 visited_mask=0x0 expected=3/0x7)  
RESULT: PAYMENT DENIED (wrong chain key). a decoy is returned
```

```
SCENARIO 4 - reorder: the checks run in the WRONG ORDER  
RESULT: PAYMENT DENIED (wrong chain key). a decoy is returned
```

When checks are skipped or their order is broken, the chain comes out different; the **real result couldn't be produced**.

This Section's Rule (9)

Don't tie critical decisions to **a single boolean** (it gets patched).

Tie security checks to a **control-flow counter** and, where possible, to a **data dependency**.



10. Response Policy and Deterrence

Why Is "Crash Immediately" Bad?

- **Response policy:** deciding **what RASP does** once it sees a threat.
- It tells the attacker "the check is exactly here."
- It makes finding and bypassing the check easier.
- Smarter responses are needed.

Examples of Good Responses

- **Delayed:** trigger the response later, somewhere else (hide the source).
- **Indirect:** silently break the feature (produce a wrong result).
- **Notify the server:** raise the risk score, reject the operation server-side.

Response Strategies — Table

Strategy	What It Does
Fail-closed	Reject the operation when in doubt, don't fall back to a trusting default
Erase the secret	Securely erase the valuable data the instant tampering is found
Decoy output	Return a random/fake result instead of crashing
Delayed/implicit response	Separate the response from the trigger, in time/distance
Device/version binding	Bind the secret to the device/version
Telemetry/attestation	Report the event to the server; let the server's risk engine decide

Designing the Response

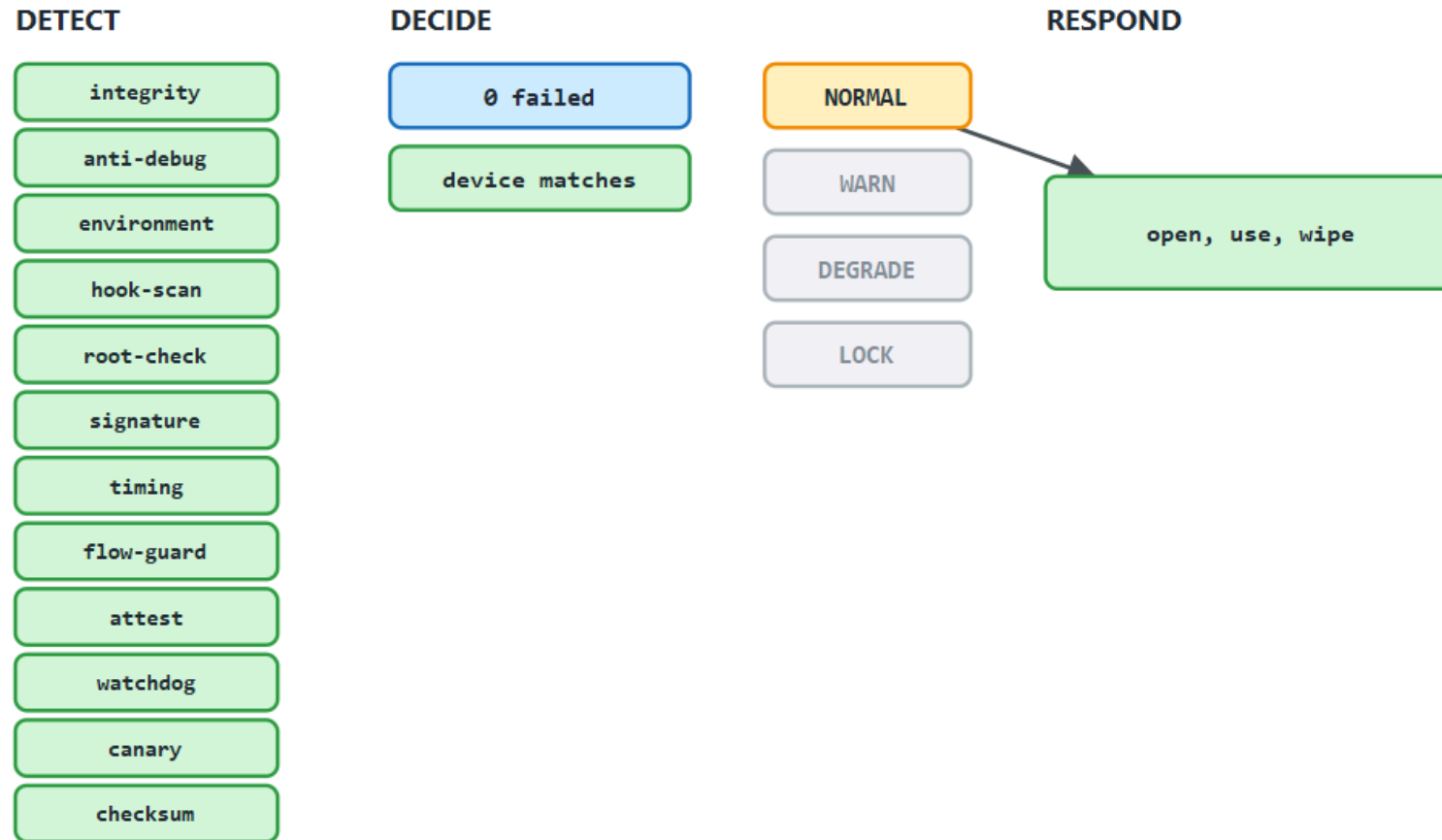
- Don't break the user experience (false-positive risk).
- A harsh response only on a **high-confidence** indicator.
- In most cases: restrict + server-side verification.

The Detect → Decide → Respond Loop

Demo 8 (`code/week-06/08-tamper-response`) combines this loop into a single **self-protection engine**:

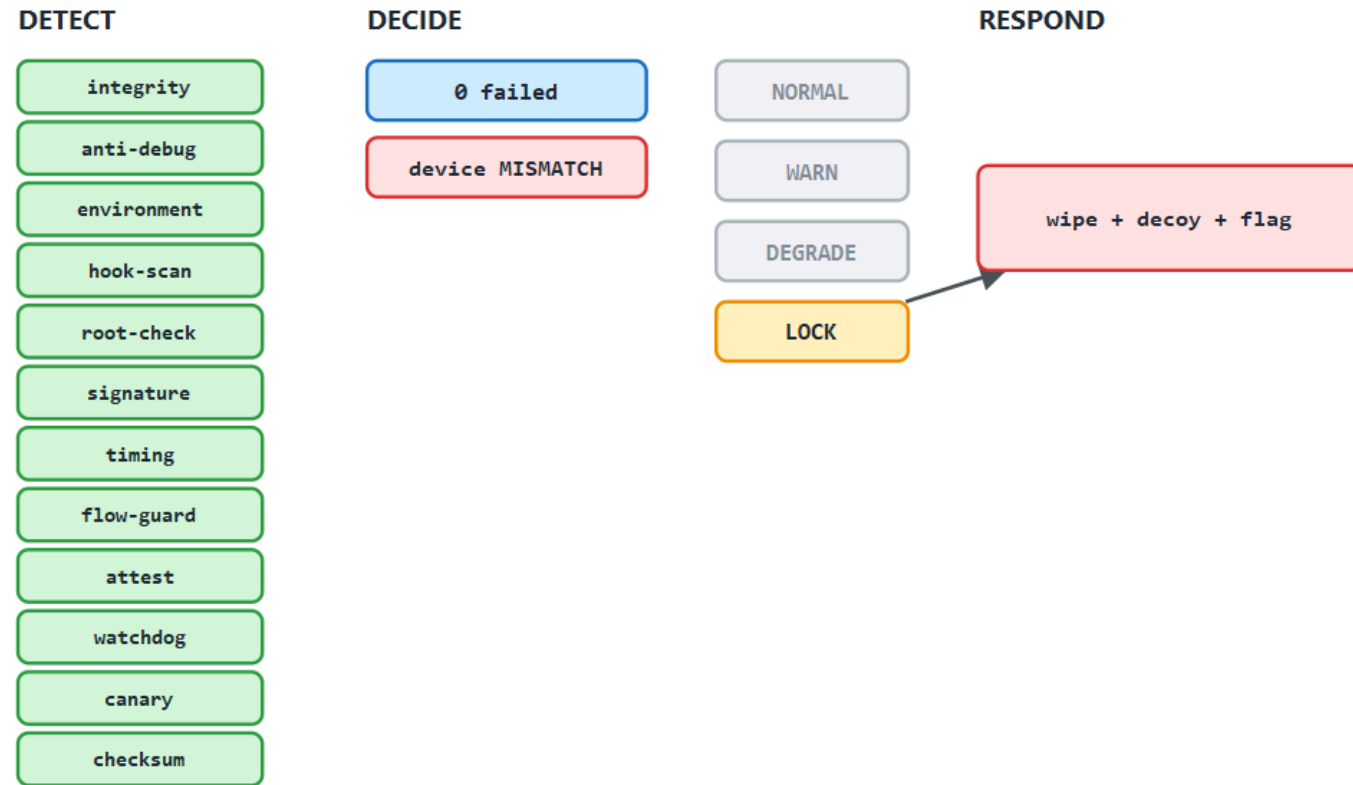
1. A series of checks **run** (integrity, anti-debug, environment...).
2. The secret is **wrapped** with a device-bound key (HKDF + AES-GCM).
3. The number of failed checks (and whether the device matches) is turned into a four-state **response policy** (`NORMAL` / `WARN` / `DEGRADE` / `LOCK`) — not a binary clean/tamper decision.

RASP Pipeline: Detect → Decide → Respond (animation)



Policy: the secret opens, is used, then wiped

RASP Pipeline — edge case: device mismatch despite passing checks



Policy: the secret is wiped; a decoy result is returned, a flag is raised

Step A · Detect (Scenario 1 — Normal)

```
SCENARIO 1 - normal: 0 failed checks, device matches -> NORMAL
RESULT: NORMAL. The secret opened and is in use
    -> "PAYMENT-KEY-7C4A"
(the secret was safely wiped from memory after use)
```

Checks **passed**, the device was correct → the secret was opened, used, and immediately **erased**.

Step B · Decide (Scenario: warn/degrade)

```
SCENARIO 2 - warn: 1 failed check -> WARN
RESULT: WARN. One weak signal is not worth degrading
         service over. The secret still opened.

SCENARIO 3 - degrade: 2 failed checks -> DEGRADE
RESULT: DEGRADE. Multiple signals -> full trust withdrawn.
RESULT-VALUE: PAYM**** (redacted)
```

One check failed → the engine **still** opened the secret, just logged it (**WARN**). Two checks failed → the secret was erased, a **redacted** result came back (**DEGRADE**).

Step C · Respond (Scenario: lock / a different device)

SCENARIO 4 - lock: 3 failed checks -> LOCK

RESULT: LOCK -> detection checks failed (tamper)

SCENARIO 5 - other-device: checks pass, device key
does not hold -> LOCK

RESULT: LOCK -> device/version binding failed (cloned?)

Even though checks passed, the device-bound key didn't match → the secret **couldn't be opened**; three failed checks lead to the same **LOCK** state. This is the innermost layer of Week 3's security shell.

Response Policy State Machine (animation)



monitoring timeline (independent calls):



summary: NORMAL=2 WARN=2 DEGRADE=2 LOCK=4

10 calls classified: 2 NORMAL, 2 WARN, 2 DEGRADE, 4 LOCK.

State Machine — edge case: always LOCK



monitoring timeline (independent calls):



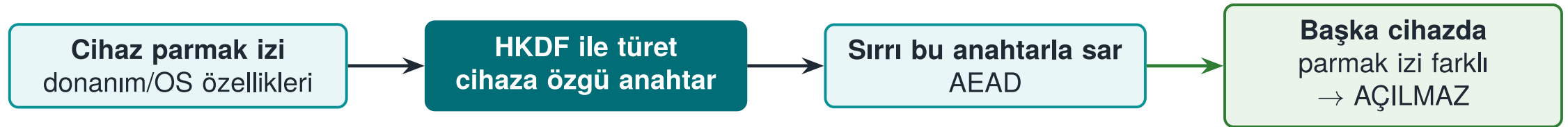
summary: NORMAL=0 WARN=0 DEGRADE=0 LOCK=10

10 calls classified: 0 NORMAL, 0 WARN, 0 DEGRADE, 10 LOCK.

Device Binding — Diagram

Cihaz bağlama: kopyalama işe yaramasın

çalınan dosya başka cihazda anlamsızdır



Code lifting ve kopyalama saldırılarının ölçeklenmesini bu engeller (11. hafta).

Device Binding

- The key/data is meaningful only on a **specific device**.
- If copied (to another device), it's useless.
- A defence against code lifting (Weeks 9/11).

The Device-Binding Flow · Step by Step

Step 1 — read the fingerprint: the device's hardware/OS fingerprint is read (Week 5).

Step 2 — derive a key: a key is derived from the fingerprint + version string with **HKDF**; the secret is wrapped with this key using **AES-GCM**.

Step 3 — if moved, try to open: even if the file is copied to another device, that device's fingerprint is **different**, so the derived key won't match → the secret **cannot be opened**.

Deterrence · the Ultimate Goal

RASP does not make the attack **impossible**; it makes it **costly and risky**.

Enough layers + server-side verification → the attacker gives up or gets caught.

Response: How Do You Decide?

1. **How reliable is the indicator?** Highly reliable → reject + notify the server. Uncertain → continue, raise the risk score.
2. **Will it affect the user experience?** If the false-positive risk is high, **don't respond harshly**; prefer silent restriction + server-side verification.
3. **Does the response give away the check?** Crashing immediately reveals the source; a **delayed/indirect** response hides it.

Response Flow · At a Glance

Tepki politikası: ne yapmalı?

tepki, algılamadan daha çok düşünülmesi gereken kısımdır

Hemen çök

saldırgana denetimin YERİNİ gösterir — kötü

Sessiz/gecikmeli tepki

sonraki işlemde sırrı sil, bağlantıyı kes

Decoy (sahte sonuç)

saldırgan yanlış yolda zaman kaybeder

Sunucuya bildir

telemetry + hesap/cihaz kısıtlama

Yanlış pozitifte meşru kullanıcıyı vurmaman, saldırıyı yormayan tepki seçilir.

This Section's Rule (10)

Separate the response from the trigger: an immediate `exit()` hands the attacker a map.

The instant tampering is found, **erase** the secret, return a **decoy** instead of crashing, report the event to the server, bind the secret to the device.

Section 8–10 — Quick Check

1. Why is a rooted device a risk?
2. Why does a counter strengthen a single `if`?
3. Why can "crash immediately" be a bad response?

Section 8–10 — Answers

1. **Root/jailbreak** removes the barriers against reading/writing memory, hooking, and certificate injection → RASP's assumptions and isolation collapse.
2. A single `if` is bypassed with a NOP/patch; the **counter** verifies after the fact that the check actually ran (distributed proof) → a single patch isn't enough.
3. "Crash immediately" shows the attacker **exactly where** the check is, and on a false positive it hits a legitimate user. A **delayed/silent/deceptive** response is better.



11. Limits and Ethics

RASP's Limits

- Every single check **can be bypassed**.
- Its strength decreases on a rooted device.
- A false positive → a real user can be hurt.
- Maintenance cost is high (tools evolve).

RASP's Limits — Diagram

RASP'ın sınırları: ne vaat eder, ne etmez?

cihazın sahibi saldırgansa yeterli zamanla her katman aşılır

ETMEZ

kırılmazlık
MATE'ye karşı kalıcı engel
kötü mimariyi kurtarma
meşru kullanıcıyı ayırt etme garantisi

EDER

saldıryı geciktirir
toplu kırılmayı zorlaştırır
telemetriyle görünürlük
riskli ortamda kapsamı daraltır

Etik sınır: kullanıcının cihazına zarar vermeyin, veri toplamayı şeffaf ve ölçülü tutun.

That's Why RASP Is...

- **Layered** (many checks), **diversified**, **hidden**.
- Combined with **server-side** verification (the real guarantee).
- Claiming it's "unbreakable" on its own is **wrong**.

Ethics and the User

- RASP runs on the user's device; **excessive data collection** is not ethical.
- A false positive can deny the user service → respond carefully.
- A balance between transparency and privacy.

The Safe-Example Rule

- Course demos **do not harm** the student's device.
- No real root/system change; synthetic indicators.
- The goal is teaching the concept, not building an attack tool.

End to End: Building a RASP Layer

Let's protect a synthetic "license check" with RASP:

- integrity checking
- debugger checking
- a smart response

Step 1 · The Point to Protect

```
int license_is_valid(void) {  
    /* ... check ... */  
    return result;    /* the attacker's target */  
}
```

This function and the flow that calls it are critical.

Step 2 · Add an Integrity Check

- After compilation, record this region's digest.
- Recompute and compare while running.
- A difference → raise the tampering flag.

Step 3 · Add Anti-Debug

- "Am I being watched?" via timing + OS indicators.
- Don't leave the result alone; **combine** it with another check.

Step 4 · Tie the Result to a Behaviour

- `license_is_valid` 's result is not plain true/false.
- The integrity + anti-debug indicators feed into the result's **computation**.
- If there's tampering, the function behaves **incorrectly**.

Step 5 · Response

- Instead of crashing immediately: report the flag to the server, restrict the feature.
- Delayed trigger: hide the source.

Step 6 · Obfuscate and Diversify

- Obfuscate the check code (Week 9).
- Place it differently in every version (diversification).
- Add a cross-check.

Step 7 · Measure and Document

- How many checks, where, and what response?
- Performance cost (checks slow things down).
- Write it up in S10.

Attacker and Defender: A Dialogue

Why This Dialogue?

Security is a **move-countermove** game.

Every defence gets an attack, every attack gets a new defence.

This is why layered defence is necessary.

Integrity · Dialogue

- **Defence:** add self-hashing.
- **Attacker:** find the digest function, force it to always say "match."
- **Defence:** obfuscate the digest check, overlap it, tie the result to a behaviour.

Anti-Debug · Dialogue

- **Defence:** add a debugger check.
- **Attacker:** find the check, bypass it.
- **Defence:** many methods + timing + obfuscation; make the response delayed.

Hook · Dialogue

- **Defence:** check the function's bytes.
- **Attacker:** hide the hook, restore the bytes.
- **Defence:** cross-checking + server-side verification.

Root · Dialogue

- **Defence:** check root indicators.
- **Attacker:** use a root-hiding tool.
- **Defence:** many indicators + a server-side risk score; restriction instead of a harsh response.

The Lesson From the Dialogue

- A single defence is always eventually bypassed.
- Strength: **multiplicity + secrecy + diversity + the server.**
- Goal: make the attack **no longer economical.**

RASP Check Catalogue (1)

Check	What It Catches	Limit
Integrity (self-hash)	A binary patch	The digest fn. can be bypassed
Anti-debug	A debugger attached	Known methods get bypassed
Emulator	A fake environment	False positives

RASP Check Catalogue (2)

Check	What It Catches	Limit
Hook/Frida	A function hook	Hidden tools
Root	Privilege escalation	Root hiding tools
Signature	A fake component	If the key leaks
CFI counter	A skipped check	A sophisticated patch

The Lesson From the Catalogue

- Every check has a **blind spot**.
- None of them is enough alone.
- Choose **several** from the catalogue, combine them, diversify them.

Choosing Checks

- How many layers, based on the asset's value?
- Which checks make sense for the platform?
- What's the performance budget?

Write the decision and its rationale in S10.

Common Mistakes

- Relying on a single check.
- Not obfuscating the check (easily found).
- A "crash immediately" response (gives away the source).
- A harsh response + a high false-positive rate (hurts the user).
- Skipping server-side verification (the real guarantee is there).



12. Project: This Week (S10)

Project · S10 (RASP + Response)

- [] At least two different RASP checks (e.g., integrity + anti-debug).
- [] Checks are **hidden** and **diversified**.
- [] Document cross-checking (Java ↔ native), if any.
- [] Document **when/where** the checks run.
- [] The response policy is **smart** (delayed/indirect — not "crash immediately").
- [] The device-binding decision and its rationale.
- [] Is there server-side verification?
- [] A false-positive plan.
- [] Limits and residual risk (what it doesn't protect).

Through the Evaluator's Eyes

- "RASP exists" isn't enough; **how many layers, how hidden, what response?**
- How does it combine with server-side verification?
- How is the false-positive risk managed?



Solved Self-Check

Question 1

What are RASP's three jobs?

Answer: Detect (abnormal environment/tampering), defend (restrict/halt the feature), deter (raise the cost).

Question 2

What does self-hashing catch, and why isn't it enough alone?

Answer: It catches a patch/tampering applied to the binary. It isn't enough alone because the attacker can find and bypass the digest function; obfuscation + cross-checking + overlapping checks are needed.

Question 3

Why is cross-checking strong?

Answer: Java checks native, native checks Java; the attacker has to bypass **both at once**.

Question 4

How does timing-based anti-debug work?

Answer: If a small piece of work took much longer than expected, someone may be single-stepping the program (a debugger).

Question 5

Why does a control-flow counter strengthen a single `if` check?

Answer: It counts that checks passed in the correct order; patching a single branch breaks the counter, and the skip is caught.

Question 6

Why isn't "crash immediately" always a good response?

Answer: It shows the attacker exactly where the check is. A delayed/indirect response + server notification is better.

Question 7

Why does the false-positive risk matter in emulator detection?

Answer: A real device/developer environment can look like an emulator; a harsh response hurts a real user.

Question 8

What is RASP's fundamental limit?

Answer: Every single check can be bypassed; it weakens on a rooted device. Strength comes from layering, diversity, and server-side verification.



Quiz-1-Style Sample Questions

Example · Multiple Choice

What is the goal of RASP cross-checking?

- A) Shrinking the code
- B) Forcing the attacker to bypass both sides ✓
- C) Speeding up encryption
- D) Turning off logging

Example · Short Answer

Name three methods that strengthen an integrity check.

Obfuscation · multiple/overlapping blocks · cross-checking · tying the result to a behaviour.

Glossary

Term	Meaning
RASP	Runtime self-protection
MATE	The attacker at the endpoint (the device's owner)
Self-hashing	Checking a digest of one's own code
Hook/Frida	Intercepting and changing a function call
CFI counter	Verifying check order by counting
Device binding	Binding data to a specific device (with HKDF)
Decoy	A fake result returned instead of crashing
Fail-closed	Rejecting an operation when in doubt
Attestation	Server-side proof of integrity

Next Week

Week 7 — Interim Project Demos (RAP1) and Week 8 — Quiz-1

In Week 7 you will present this week's RASP checks (integrity checking, debugger/environment/hook detection, the control-flow counter, the response policy) live in your project. Week 8 is Quiz-1, covering weeks 1–6.

Then **Week 9 — Advanced Obfuscation and Diversification** continues by deepening this week's debugger/environment detection ideas and its "obscurity/deterrence is not unbreakability" theme on the code-obfuscation side.

This week in one sentence: RASP monitors itself while the program runs and responds to threats; it gains its strength not from a single check but from **layered, diversified, hidden** checks and **server-side** verification. The real guarantee: layered RASP + obfuscation + short-lived keys + **server-side verification** — a **time** game played against the attacker.